

Cardiff School of Computer Science and Informatics

Coursework Assessment Pro-forma

Key Information

Module Code:	CM2104
Module Title:	Computational Mathematics
Module Leader:	Prof. Paul Rosin
Module Moderator:	Dr. Hiroyuki Kido
Assessment Title:	Plotting Circles
Assessment Number:	1
Assessment Weighting:	50% of a 10-credit level 2 module
Assessment Limits:	A MATLAB program with GUI plus a report (1–2 pages of text; plus diagrams, screenshots etc.).

The Assessment Calendar can be found under ‘Assessment & Feedback’ in the COMSC-ORG-SCHOOL organisation on Learning Central. This is the single point of truth for (a) the hand out date and time, (b) the hand in date and time, and (c) the feedback return date for all assessments.

Learning Outcomes

The learning outcomes for this assessment are as follows:

- Show a clear understanding of basic MATLAB programming environment and data structures
- Understand the practical implementation of some general mathematical techniques via MATLAB
- Demonstrate an awareness of basic Linear Algebra and its application to Computational Geometry concepts with MATLAB

Submission Instructions

The coversheet can be found under ‘Assessment & Feedback’ in the COMSC-ORG-SCHOOL organisation on Learning Central.

Files should be submitted via Learning Central. The submission page can be found under ‘Assessment & Feedback’ in the CM2104 module on Learning Central. Your submission should consist of multiple files:

Description		Type	Name
<u>Cover sheet</u>	Compulsory	One PDF (.pdf) file	[Student number].pdf
<u>Report</u>	Compulsory	One PDF (.pdf) file	Report_[Student number].pdf
<u>Source code</u>	Compulsory	Matlab files for the GUI (.mlapp) and modified function stubs (.m) packaged as a single zip file	Code_[Student number].zip

Any deviation from the submission instructions above (including the number and types of files submitted) may result in a reduction in marks for the assessment.

Any code submitted will be run on a system equivalent to those available in the Linux laboratory, should be able to run on the School’s MATLAB installation (**without requiring the use of any MATLAB toolboxes**), and must be submitted as stipulated in the instructions above.

If you are unable to submit your work due to technical difficulties, please submit your work via e-mail to comsc-submissions@cardiff.ac.uk and notify the module leader.

Assessment Description

Develop a graphical user interface (GUI) implemented in MATLAB using App Designer that allows the user to specify and draw several circles according to some geometric construction rules.

To enable me to carry out unit testing outside of your GUI, for some of the tasks below a function stub is provided (download `stubs.zip` which contains them) which you should modify to include the appropriate functionality. For your convenience, when you integrate these functions into your GUI you can modify them.

Another requirement is that you should use **homogeneous coordinates** to represent all the data.

Please note: You are allowed to use any of John Burkardt's Geometric Processing toolbox functions (a link was provided in the slides) and third-party libraries, as long as the third-party add-ons run on their own without any initial set-up. You must clearly reference the sources in your report.

Task 1 (5% weight): Write a MATLAB GUI that allows the user to enter points, which are then displayed in the window. Note: all points input by the user should be entered by clicking locations in the window using the mouse (rather than entering coordinates in a text box). The GUI should also provide buttons to allow the user to hide or reveal the various geometric elements that will be drawn during the coursework (i.e. hide/reveal all points, hide/reveal all lines, hide/reveal all intermediate constructions).

Extend the above GUI so that when the “**run construction**” button is pressed by the user the following steps for Tasks 1–6 are carried out in sequence (i.e. draw the geometric elements as soon as they are fully specified):

Task 2 (5% weight): Construct and draw the first circle

The user enters a point which is the centre of the circle. The user enters another point, which will lie on the circumference of the circle. Draw the points and the circle.

Task 3 (5% weight): Construct and draw the second circle

The user enters a point, which is the centre of the circle. The second circle should have tangential contact with the first circle. Draw the point and the circle. Modify this function stub to compute the radius of the second circle: `radiusTangentialCircle`

Task 4 (30% weight): Construct and draw the third circle

Enter 2 points and for each find the closest point on the closest circle (i.e. “snap” to the circle). These two closest points will lie on the circumference of the third circle. In more detail:

- The user enters the first point; determine which circle it is closest to, and find the closest point on its circumference.
- The user enters another point; repeat the process to find the closest point. Report an error if both user points are closest to the same circle.
- Modify this function stub to find which circle is closest and also the closest point: `closestPointsOnCircle`

Select another point on the third circle

- Construct and draw the perpendicular bisector of the line segment between the centres of circles 1 and 2.

- Modify this function stub to determine the parameters of the perpendicular bisector: `centreBisector`
- The user enters a point. Determine the closest point on the perpendicular bisector to this point (i.e. “snap” to the line); that closest point will be another point on the circumference of the third circle. Modify this function stub: `closestPointBisector`

Determine the parameters of the third circle

- Construct the two perpendicular bisectors through the line segments between the snapped points 1 & 2 and 1 & 3.
- The circle centre is at the intersection of the perpendicular bisectors.
- Modify this function stub to determine the circle centre and radius: `circleSolve`

Plot the initial user selected points, the calculated closest points, the perpendicular bisectors, and the third circle.

Task 5 (5% weight): Circle intersections

Find the intersections of the third circle with the first two circles, and plot them. Modify this function stub to determine the circle intersections: `intersections2circles`

Task 6 (10% weight): Geometric transformation

Consider the translation that will place the centre of the third circle at the origin. Consider the rotation about the origin that will align the perpendicular bisector of the line segment between the centres of circles 1 and 2 with the X-axis, with the constraint that the rotation angle should be in the range $[0, 180^\circ]$. Calculate the translation and rotation matrices and an overall combined transformation matrix (all using homogeneous coordinates). Apply the transformation to all the geometric elements on the window (i.e. translate and rotate the circles, points, etc. in the window) Modify this function stub to determine the matrices: `transformationMatrices`

Task 7 (10% weight): Save and replay option

The GUI should also allow the user to save all the points inputted by the users in a file. Use the following data format for the saved coordinates: a text file containing the ordered, space separated x and y coordinates, one pair per line. They should be ordered in the file in the same order as they are entered by the user. The GUI should allow the user to load a set of points from such a file and then directly construct the circles and the additional geometric elements and display them.

Task 8 (20% weight): In order to gain higher marks you need to add some novel extensions or additional features. You need only provide **two** further different **novel** extensions (such as those suggested here). There are endless possibilities here and you are encouraged to think of your own extensions. Here are a few suggestions:

- Allow the user to select and modify elements in the window, after which the remaining elements are automatically updated.
- The third circle is replaced by a sphere, for which the user has some geometric means of specifying its parameters. The intersections of the sphere with the first two circles are found and plotted.
- The GUI includes some form of animation of the geometric elements in the window.

Task 9 (10% weight): You must supply a report on your submission which provides a short written description (1–2 pages of text; plus diagrams, screenshots etc.) conveying all the appropriate information to demonstrate its operation and explaining your extension of the basic algorithm. Include your student number in the report.

Assessment Criteria

Credit will be awarded according to the correct functioning of the following components of the code.

1. 5% – Basic MATLAB GUI (Task 1)
2. 5% – Construct and draw the first circle (Task 2)
3. 5% – Construct and draw the second circle (Task 3)
4. 30% – Construct and draw the third circle (Task 4)
5. 5% – Circle intersections (Task 5)
6. 10% – Geometric transformation (Task 6)
7. 10% – Save and replay option (Task 7)
8. 20% – Design and implementation of novel extensions or additional features (Task 8)
9. 10% – Report describing the operation of your program and your extension of the basic algorithm (Task 9)

High 80%+	1st	• Task 1: Exceptional GUI design that allows seamless user interaction, with intuitive controls for hiding/revealing geometric elements. All geometric elements are displayed with clarity and precision. • Task 2: Masterful implementation of circle construction with precise user input handling and flawless graphical representation. • Task 3: Effective approach to tangential circle construction, demonstrating profound understanding of geometric principles and flawless execution. • Task 4: Detailed and accurate implementation of the closest point calculations and perpendicular bisector construction, with clear visual representation and error handling. • Task 5: Comprehensive and accurate intersection calculations with clear graphical output, demonstrating advanced understanding of circle geometry. • Task 6: Precise calculation and application of transformation matrices, resulting in seamless visual updates. • Task 7: Effective file handling for saving and loading points, with clear user feedback and error handling. • Task 8: Two novel extensions that significantly enhance functionality, demonstrating creativity and advanced programming skills. • Task 9: Comprehensive and well-structured report with clear explanations, diagrams, and screenshots, effectively conveying the operation and extensions of the GUI.
--------------	-----	--

1st 70-79%	• Task 1: Well-designed GUI that allows user-friendly point entry and effective controls for geometric elements. • Task 2: Accurate circle construction with clear graphical output and user input validation. • Task 3: Effective implementation of tangential circle construction with minor improvements needed in user interaction. • Task 4: Detailed implementation of closest point calculations and perpendicular bisector construction, with some minor omissions in clarity or execution. • Task 5: Accurate intersection calculations with clear output, though some minor details may be lacking. • Task 6: Effective transformation matrix calculations with minor issues in application to geometric elements. • Task 7: Functional file handling for saving and loading points, with some minor usability issues. • Task 8: Two extensions that enhance functionality and demonstrating good programming skills, though lacking in innovation compared to high first-class work. • Task 9: Clear report with adequate explanations and diagrams, effectively conveying the operation of the GUI, with only minor omissions or clarity issues.
------------	---

2.i 60-69%	• Task 1: Functional GUI design, but with noticeable usability issues or lack of clarity. • Task 2: Accurate circle construction, but with some inaccuracies in representation or minor graphical representation flaws. • Task 3: Feasible implementation of tangential circle construction, but with evident flaws or lacking depth in user interaction. • Task 4: Adequate implementation of closest point calculations, but with noticeable flaws in execution. • Task 5: Basic intersection calculations with some inaccuracies in graphical output. • Task 6: Adequate transformation matrix calculations, but with inconsistencies in application or inaccuracies in calculations. • Task 7: Basic file handling for saving and loading points, but with significant usability issues. • Task 8: One relevant extension that adds some functionality, but lacks depth or innovation. • Task 9: Adequate report with some explanations and diagrams, but lacks detail, depth or clarity.
2.ii 50-59%	• Task 1: Basic GUI design that allows point entry, but with significant usability issues. • Task 2: Basic circle construction with evident deficiencies in accuracy or noticeable flaws in graphical representation. • Task 3: Incomplete or flawed tangential circle construction • Task 4: Basic implementation of closest point calculations, but with considerable flaws in execution. • Task 5: Incomplete intersection calculations with significant inaccuracies. • Task 6: Limited understanding of transformations, with significant errors. • Task 7: Incomplete save/load functionality with major usability flaws. • Task 8: No extensions or very basic attempts that do not enhance functionality. • Task 9: Limited report with minimal explanations and diagrams, lacking clarity.

3rd 40-49%	• Task 1: Limited progress on GUI design, with major usability flaws. • Task 2: Minimal progress on circle construction, with significant inaccuracies. • Task 3: Very limited attempt at tangential circle construction, largely incomplete. • Task 4: Little progress on closest points, with major flaws in execution. • Task 5: Minimal attempt at intersection calculations, largely inaccurate. • Task 6: Very limited understanding of transformations, with major errors. • Task 7: Little or no progress on save/load functionality. • Task 8: No extensions provided or irrelevant extensions that do not enhance functionality. • Task 9: Poorly written report with little explanation or diagrams, lacking coherence.
Marginal Fail 30-39%	• Task 1: Little progress toward a functional GUI, with severe usability issues. • Task 2: Very limited circle construction with major flaws. • Task 3: Incomplete or inappropriate implementation of tangential circle construction. • Task 4: Little progress on closest point calculations, with significant flaws. • Task 5: Very limited progress on intersection calculations with major inaccuracies. • Task 6: Very limited transformation matrix calculations with inappropriate application. • Task 7: Extremely limited file handling capabilities, with major issues. • Task 8: No relevant extensions provided. • Task 9: Very poorly written report with minimal information.

Fail 0-29%	• Task 1: No progress toward a functional GUI, with severe usability issues. • Task 2: No progress on circle construction. • Task 3: No implementation of tangential circle construction. • Task 4: No progress on closest point calculations. • Task 5: No progress on intersection calculations. • Task 6: No transformation matrix calculations. • Task 7: No file handling capabilities. • Task 8: No extensions provided. • Task 9: No report submitted or report lacking any relevant information.
------------	---

Help and Support

Questions about the assessment can be asked in weeks 7 to 10 of the Autumn Semester either in person or via e-mail. Responses will be provided either in person, via e-mail or via Learning Central, and may be anonymised and broadcast if the response to the question would benefit other students. I will aim to respond to questions within two working days (responses may take longer than two working days during reading week, or during Christmas / Easter recess).

Feedback

Feedback on your coursework will address the above assessment criteria. Feedback and marks will be returned via e-mail or via Learning Central.